Práctica 7

Plan de Aseguramiento de Calidad de Software (SQAP) con IEEE Std 730

Modalidad: grupal, 3 a 4 estudiantes. SQAP mínimo guiado de un proyecto piloto (Parte A) + auditoría de un SQAP con defectos sembrados (Parte B) + SQAP completo de 16 secciones en cuatro entregables (Parte C).

Duración estimada: 2 periodos de laboratorio para las Partes A y B; 4 semanas para la Parte C.

Requisito: haber completado las Prácticas 1 a 6. Prerrequisito de la carrera: Arquitectura de Software (atributos de calidad, tácticas, escenarios). Word o Google Docs, Excel o Google Sheets, Git y una cuenta de GitHub.

Material: `practica7-material.zip` con la plantilla del SQAP, el plan con defectos `SQAP-BIBLIOUSFX-v0.9.docx`, el libro `matrices-sqap.xlsx` y la plantilla de reporte de defectos.

Entregable: SQAP mínimo (Parte A) y tabla de auditoría (Parte B) en PDF; SQAP completo en DOCX y PDF, versionado en la carpeta `docs/` del repositorio del proyecto piloto, y defensa oral de 20 minutos (Parte C).

1. Objetivos

Al terminar la práctica el estudiante será capaz de:

- Explicar qué es un Plan de Aseguramiento de Calidad de Software (SQAP), en qué se diferencia de un plan de pruebas y de un plan de proyecto, y por qué se escribe antes de construir y no al final.
- Estructurar un SQAP conforme a las 16 secciones de IEEE Std 730-2002, agrupándolas en las tres áreas de IEEE Std 730-2014: implementación del proceso SQA, aseguramiento del producto y aseguramiento del proceso.
- Pasar de un requerimiento no funcional a una métrica verificable: característica de ISO/IEC 25010, métrica, fórmula, instrumento, momento de medición y valor objetivo.
- Asociar cada métrica del plan con la herramienta de las Prácticas 1 a 6 que la mide (JUnit, JaCoCo, SonarQube, Cypress, JMeter, Jenkins) y escribir el mismo umbral en el plan y en la configuración de la herramienta.
- Planificar revisiones, inspecciones y auditorías, una estrategia de pruebas por nivel con criterios de entrada y salida, y un flujo de gestión de defectos con severidad, prioridad y SLA coherentes.
- Construir una matriz de riesgos de calidad con probabilidad, impacto, exposición y mitigaciones accionables.
- Auditar un SQAP ajeno contra una regla explícita de verificabilidad, localizar sus defectos, justificar cada uno con la evidencia textual y corregirlo con una entrada en el historial de versiones del plan.

2. Requisitos previos

Recurso	Versión / detalle	Para qué se usa
Prácticas 1 a 6	Completadas, con sus proyectos a mano	Son los instrumentos que el plan pone a trabajar: el SQAP dice qué medir y con qué umbral; las prácticas ya enseñaron cómo.

Recurso	Versión / detalle	Para qué se usa
Arquitectura de Software	Prerrequisito de la carrera	Atributos de calidad, escenarios y tácticas: el SQAP verifica las decisiones de arquitectura.
Procesador de texto	Word, LibreOffice Writer o Google Docs	Redactar el SQAP a partir de `plantilla-SQAP.docx`.
Hoja de cálculo	Excel o Google Sheets	Métricas, riesgos y componentes de terceros en `matrices-sqap.xlsx` (la exposición se calcula sola).
Git y GitHub	Git 2.40 o superior	El SQAP se versiona en `docs/` del repositorio del proyecto piloto, como cualquier otro artefacto.
Diagramas	draw.io, Mermaid o PlantUML	Organigrama, flujo de estados de un defecto, ubicación de las revisiones en el ciclo de vida.
Normas	IEEE Std 730, ISO/IEC 25010	Acceso por la biblioteca de la universidad o IEEE Xplore. El Anexo A resume lo indispensable.

> **Sobre el trabajo en grupo:** el SQAP es un documento de equipo, pero cada sección tiene un autor. Anoten en la Sección 16 (historial) quién escribió y quién revisó cada sección. En la defensa oral el docente puede preguntar a cualquier integrante sobre cualquier sección.

3. El plan de calidad: qué es y dónde encaja

En las Prácticas 1 a 6 aprendiste a medir: cuántas pruebas pasan, qué cobertura alcanzan, cuántos code smells hay, cuánto tarda el percentil 95 bajo carga, si el build está sano. Pero en cada práctica alguien te dio el umbral: 80 % de cobertura, Quality Gate en verde, p95 por debajo del SLA. **Esta práctica responde la pregunta que quedó abierta: ¿quién decide esos números, con qué criterio y dónde quedan escritos?** La respuesta es el plan de calidad.

Un Plan de Aseguramiento de Calidad de Software, conocido por su sigla inglesa **SQAP** (Software Quality Assurance Plan), es el documento que establece qué compromisos de calidad asume un proyecto, cómo se va a comprobar cada uno, quién lo comprueba, cuándo y dónde queda la evidencia. No es un plan de pruebas (que solo cubre la verificación ejecutable) ni un plan de proyecto (que se ocupa de alcance, tiempo y costo): el SQAP orquesta todas las actividades de aseguramiento a lo largo del ciclo de vida.

> **Principio rector:** IEEE Std 730-2014 plantea la calidad como algo que se diseña y se planifica desde el inicio, desplazando la prevención y detección de defectos hacia las fases tempranas, donde corregirlos cuesta mucho menos. Según los datos clásicos de Boehm, un defecto de requerimientos corregido en producción puede costar hasta 100 veces más que corregido durante el análisis.

Qué cambia con un plan de calidad

Pregunta	Sin SQAP	Con SQAP
¿Qué cobertura es suficiente?	La que salga, o la que diga la herramienta por defecto.	La que el plan fija para cada módulo, justificada por su criticidad.

Pregunta	Sin SQAP	Con SQAP
¿Quién revisa el código crítico?	Quien tenga tiempo; a veces su propio autor.	Un rol definido, independiente de quien lo escribió.
¿Cuándo se prueba el rendimiento?	Al final, si sobra tiempo.	En un punto de control acordado, con la carga del requerimiento.
¿Qué es un defecto crítico?	Lo que el más ruidoso del equipo diga.	Una definición escrita, con tiempo de respuesta y de resolución.
¿Qué pasa si se va un integrante?	Se improvisa.	Un riesgo identificado de antemano, con mitigación y responsable.

Cómo encaja con las prácticas anteriores

Cada práctica anterior hizo cumplir un número que el SQAP debe fijar. El plan es el documento rector; las herramientas son su brazo ejecutor:

Práctica	Qué automatiza	Sección del SQAP que fija el criterio
1 y 2 — JUnit y TDD	Pruebas unitarias, diseño guiado por pruebas	Sección 7 (Pruebas): nivel unitario, cobertura mínima, convención de nombres. Sección 9: TDD obligatorio en módulos críticos.
3 — SonarQube	Análisis estático y Quality Gate	Sección 5 (Métricas): umbrales de deuda técnica, duplicación, cobertura y vulnerabilidades.
4 — Cypress	Pruebas E2E en navegador	Sección 7: escenarios de aceptación automatizables y su criterio de salida.
5 — JMeter	Rendimiento contra un SLA	Sección 5: objetivos de eficiencia de desempeño (percentiles, carga, tasa de error).
6 — Jenkins	Integración continua	Sección 9 (Herramientas): el pipeline como mecanismo que hace cumplir el plan en cada cambio.
8 — CI/CD	Despliegue continuo	Secciones 10 y 12: control del repositorio, ambientes y evidencias que se conservan.

El estándar: IEEE Std 730

El estándar internacional que rige la estructura y el contenido del SQAP es el IEEE Std 730, de la IEEE Computer Society. Se usan dos ediciones complementarias:

Edición	Enfoque	Uso en la práctica
IEEE 730-2002	Plantilla prescriptiva de 16 secciones. Didáctica y autoexplicativa.	Esqueleto del documento a entregar.

Edición	Enfoque	Uso en la práctica
IEEE 730-2014	Marco de procesos armonizado con ISO/IEC/IEEE 12207 e ISO/IEC/IEEE 15289. Tres áreas: implementación del proceso SQA, aseguramiento del producto y aseguramiento del proceso.	Marco conceptual para agrupar las secciones y justificar decisiones.

Las 16 secciones de IEEE 730-2002 se resumen en el **Anexo A**. Si una sección no aplica al proyecto, se escribe "Esta sección no aplica a este plan" junto con la justificación; **nunca se omite**. El SQAP se apoya además en otras normas que ya conoces:

- **ISO/IEC 25010** — modelo de calidad del producto. Reemplazó a ISO/IEC 9126; en su edición 2011 define ocho características (adecuación funcional, eficiencia de desempeño, compatibilidad, usabilidad, fiabilidad, seguridad, mantenibilidad y portabilidad). La edición 2023 agrega la protección (safety) y renombra usabilidad y portabilidad como capacidad de interacción y flexibilidad. El Anexo B lista ejemplos de métricas.
- **ISO/IEC/IEEE 12207** — procesos del ciclo de vida en los que se insertan las actividades SQA.
- **ISO/IEC 29110** — perfil para entidades muy pequeñas (VSE), aplicable a equipos estudiantiles y startups.
- **CMMI-DEV** — áreas de proceso PPQA, VER y VAL (en CMMI 2.0: PQA y VV). **TMMi** — madurez del proceso de pruebas.
- **IEEE Std 1028** — revisiones técnicas, inspecciones, walkthroughs y auditorías.
- **ISO/IEC/IEEE 29119** — documentación y procesos de prueba (reemplaza a IEEE 829).
- **SWEBOK V4 (2024)** — capítulo 12, Software Quality.

De un requerimiento a una métrica verificable

El corazón del SQAP es una cadena que no admite eslabones sueltos. Si un eslabón falta, el compromiso no se puede comprobar y, en la práctica, no existe:

Eslabón	Pregunta	Ejemplo (BIBLIOUSFX, búsqueda de libros)
Requerimiento	¿Qué pidió el cliente?	RNF-01: la búsqueda debe responder rápido en horario pico.
Característica (25010)	¿Qué cualidad del producto es?	Eficiencia de desempeño → comportamiento temporal.
Métrica	¿Qué número la representa?	Tiempo de respuesta, percentil 95, de `GET /api/libros?q=`.
Instrumento y condiciones	¿Con qué y en qué escenario se mide?	JMeter (Práctica 5), 100 usuarios concurrentes, 10 minutos, ambiente de staging.
Momento	¿Cuándo se mide?	En cada release candidate, como etapa del pipeline (Práctica 6).
Valor objetivo	¿Qué número es aceptable?	p95 ≤ 800 ms y tasa de error < 1 %.

Eslabón	Pregunta	Ejemplo (BIBLIOUSFX, búsqueda de libros)
Responsable y evidencia	¿Quién lo comprueba y dónde queda?	Tester; dashboard HTML de JMeter archivado en el build.

La regla de la práctica: qué es un plan verificable

Así como en la Práctica 6 "verde no alcanzaba" y había que acordar qué era un build sano, aquí **un documento completo no alcanza**. Un SQAP puede tener las 16 secciones, buena redacción y tablas prolijas, y aun así no comprometer a nada. Un plan verificable cumple las siete condiciones siguientes; contra ellas se evalúan los dos planes de la práctica:

Indicador	Condición	Cómo se comprueba
Completo	Están las 16 secciones de IEEE 730-2002; las que no aplican lo dicen y lo justifican.	Comparar el índice con el Anexo A.
Medible	Cada objetivo de calidad tiene métrica, fórmula y valor objetivo numérico o verificable.	Ningún objetivo dice "rápido", "intuitivo", "alto" o "adecuado" sin un número.
Instrumentado	Cada métrica dice con qué herramienta o procedimiento, en qué condiciones y en qué momento se mide.	Para cada métrica se puede nombrar la práctica (1 a 6) o el procedimiento que la mide.
Asignado e independiente	Cada actividad tiene responsable, y quien verifica un artefacto no es quien lo produjo.	Cruzar la tabla de roles (Sección 3) con las revisiones (Sección 6).
Consistente	Un mismo umbral tiene el mismo valor en todas las secciones y en la configuración de las herramientas.	Buscar cada número (80 %, 800 ms…) en todo el plan, el Quality Gate y el `Jenkinsfile`.
Trazable	Cada métrica se vincula a un requerimiento o a un riesgo; cada riesgo de exposición alta tiene mitigación accionable.	Columna RNF/Riesgo de las métricas; matriz de riesgos (Sección 14).
Evidenciable	Cada actividad deja un registro con ubicación, formato y período de retención.	Sección 12: catálogo de evidencias.

PARTE A — Práctica guiada: el SQAP mínimo de tu proyecto piloto

En esta parte el equipo escribe, durante la clase, las secciones del SQAP que deciden todo lo demás: propósito, roles, métricas, pruebas, defectos y riesgos. Cada paso muestra primero el ejemplo conductor (**BIBLIOUSFX**, el sistema de biblioteca de la Opción A) y después pide lo mismo para tu proyecto. El ejemplo ilustra el nivel de detalle esperado; **no se copia**.

Paso 1. El material

Descomprime `practica7-material.zip`. Contiene:

Archivo	Qué contiene
`plantilla-SQAP.docx`	Las 16 secciones de IEEE 730-2002 con lo que debe responder cada una y las tablas vacías. Es la base del SQAP de la Parte A y de la Parte C.
`matrices-sqap.xlsx`	Cuatro hojas: Métricas, Riesgos (la exposición y el nivel se calculan solos), SLA de defectos y Componentes de terceros. Trae filas de ejemplo de BIBLIOUSFX que debes reemplazar.
`plantilla-reporte-defecto.md`	Formato de reporte de defectos para la Sección 8.
`reto/SQAP-BIBLIOUSFX-v0.9.docx`	El plan con defectos sembrados de la Parte B. No lo abras todavía.

Paso 2. Elegir el proyecto piloto y delimitar el alcance (Sección 1)

El SQAP se aplica a un proyecto concreto. Cada equipo elige uno de estos sistemas (o propone otro de tamaño similar, con aprobación del docente):

Opción	Descripción	Stack sugerido
A. Biblioteca universitaria	CRUD web con autenticación, préstamos, multas y reportes. Roles: estudiante, docente, bibliotecario, administrador.	Java + Spring Boot + PostgreSQL + React, o PHP + Laravel + MySQL
B. API de inventario de laboratorio	Catálogo de equipos, reservas, reporte de daños, trazabilidad. Documentación OpenAPI.	Node.js + Express + MongoDB, o Spring Boot + PostgreSQL
C. Control de asistencia móvil	App con escaneo QR o geolocalización, backend de reportes y panel administrativo.	Flutter o Android nativo + Firebase, o backend propio en Node o Spring
D. Registro de calificaciones	Web para docentes con cálculo de promedios, exportación a PDF, historial y auditoría.	Laravel + MySQL, o Django + PostgreSQL
E. Punto de venta (POS)	Venta, inventario, cierre de caja, reportes diarios. Multiusuario con permisos.	Laravel + MySQL, o Django + PostgreSQL

Delimita el alcance del plan, no el del sistema entero: entre **2 y 4 historias de usuario o casos de uso principales**, al menos dos roles de usuario y atributos de calidad que se puedan medir con las herramientas de las Prácticas 1 a 6. Luego clasifica el software según su nivel de integridad, con el esquema de cuatro niveles de IEEE Std 1012:

Nivel	Consecuencia de una falla	Ejemplo
4 — Alto	Pérdida de vidas, daño físico o ambiental grave.	Control de dosis de un equipo médico.
3 — Mayor	Pérdida económica importante, incumplimiento legal, falla de la misión.	Facturación, pagos, datos personales sensibles.
2 — Moderado	Inconveniencia operativa recuperable.	Biblioteca, asistencia, inventario de laboratorio.

Nivel	Consecuencia de una falla	Ejemplo
1 — Menor	Molestia sin consecuencias apreciables.	Sitio informativo interno.

Ejemplo conductor. “Este plan cubre el desarrollo de BIBLIOUSFX v1.0, aplicación web de gestión de biblioteca de la Facultad de Ciencia y Tecnología. Abarca requerimientos, diseño, construcción y pruebas de los casos de uso Buscar libro, Registrar préstamo, Registrar devolución y Calcular multa, durante la Gestión II/2026, por el equipo M-07. Se clasifica en el nivel de integridad 2: su falla causa inconveniencia operativa, pero no pérdida de vidas ni daño económico mayor.”

Escribe la Sección 1 de tu proyecto con la misma estructura: **Evidencia 1**.

Paso 3. Roles, independencia y RACI (Sección 3)

Asigna los roles del equipo. En un equipo de tres o cuatro personas cada uno tendrá más de un rol; lo que no se negocia es la **independencia**: quien verifica un artefacto crítico no puede ser quien lo produjo. Si el Responsable SQA también programa, no programa los módulos críticos.

Rol	Responsabilidad principal	BIBLIOUSFX
Líder de proyecto	Coordinación, cronograma, comunicación con el docente.	Integrante 1
Arquitecto	Decisiones de arquitectura (ADR), revisión de diseño.	Integrante 2
Responsable SQA	Plan SQA, métricas, auditorías, aprobación de revisiones de los módulos críticos.	Integrante 3 (desarrolla solo el módulo de reportes)
Desarrollador backend	API y pruebas JUnit; módulos Préstamos y Multas.	Integrantes 1 y 2
Desarrollador frontend / tester	Interfaz, pruebas Cypress y JMeter.	Integrante 4

Completa una matriz **RACI** con las actividades de calidad en las filas (redactar el SQAP, revisar el SRS, revisar código de módulos críticos, ejecutar pruebas de carga, aprobar una release, auditar el cumplimiento del plan) y los roles en las columnas. Cada fila tiene exactamente una **A** (Accountable, quien rinde cuentas) y al menos una **R** (Responsible, quien la ejecuta). **Evidencia 2**.

Paso 4. De atributo de calidad a métrica (Sección 5)

Elige al menos **tres** características de ISO/IEC 25010 relevantes para tu proyecto y recorre para cada una la cadena de la Sección 3: requerimiento, característica, métrica, fórmula, instrumento, momento y objetivo. Usa la hoja **Métricas** de `matrices-sqap.xlsx`. El ejemplo conductor prioriza cinco:

Característica	Justificación	Métrica e instrumento	Objetivo
Adecuación funcional	El cálculo de multas no admite errores.	% de casos de prueba de Multas que pasan (JUnit, en cada build).	100 %
Eficiencia de desempeño	Uso concurrente en horario pico.	p95 de la búsqueda con 100 usuarios, 10 min (JMeter, en cada release).	≤ 800 ms, error < 1 %

Característica	Justificación	Métrica e instrumento	Objetivo
Usabilidad	Usuarios nuevos cada semestre.	Tasa de éxito de la tarea “registrar un préstamo” sin ayuda, 5 usuarios nuevos; cuestionario SUS.	≥ 90 %; SUS ≥ 68
Seguridad	Datos personales de estudiantes.	Vulnerabilidades altas o críticas (SonarQube y Dependabot, en cada PR).	0
Mantenibilidad	El proyecto sigue en gestiones posteriores.	Cobertura de líneas de Préstamos y Multas (JaCoCo); deuda técnica (SonarQube).	≥ 80 %; < 5 días-persona

> **La prueba del “¿cómo lo sabrías?”:** para cada objetivo pregúntate: si mañana el docente dice que no se cumple, ¿qué número le mostrarías y de qué herramienta sale? Si la respuesta es “nos parece que sí”, no es una métrica. “La interfaz será intuitiva” no se puede medir; “el 90 % de cinco usuarios nuevos registra un préstamo sin ayuda” sí.

Como referencia, así se ve un atributo bien desglosado en subcaracterísticas (Fiabilidad):

Subcaracterística	Métrica	Fórmula / instrumento	Objetivo
Madurez	Densidad de defectos en producción	Defectos / KLOC en los primeros 30 días	≤ 1,0 def/KLOC
Disponibilidad	Disponibilidad mensual	(tiempo total − tiempo caído) / tiempo total; monitor externo cada 5 min	≥ 99,0 %
Tolerancia a fallos	Tasa de recuperación	% de operaciones que reintentan con éxito	≥ 95 %
Recuperabilidad	RTO (Recovery Time Objective)	Tiempo máximo de restauración tras una falla	≤ 30 minutos

Evidencia 3: la hoja Métricas con al menos tres características y dos métricas por característica.

Paso 5. De la métrica a la herramienta: el mismo número en dos lugares

Un umbral que solo está en el plan es una intención. Un umbral que solo está en la herramienta es una configuración que nadie justificó. **El mismo número tiene que estar en los dos lugares.** Para cada métrica del Paso 4 que mide una herramienta de las Prácticas 1 a 6, escribe dónde se configura ese umbral:

Métrica del plan	Dónde se hace cumplir	Configuración
Cobertura de líneas ≥ 80 %	JaCoCo en el `pom.xml` (Práctica 6, Ejercicio 2)	`<minimum>0.80</minimum>` en la regla `check`
Cobertura en código nuevo ≥ 80 %	Quality Gate de SonarQube (Práctica 3)	Condición Coverage on New Code is less than 80.0

Métrica del plan	Dónde se hace cumplir	Configuración
p95 ≤ 800 ms, error < 1 %	Plan de JMeter (Práctica 5) en una etapa del pipeline	Duration Assertion o validación del `summary`, ejecutado con 100 hilos
0 vulnerabilidades altas	Quality Gate de SonarQube y Dependabot	Security Rating on New Code worse than A
Build sano en cada cambio	`Jenkinsfile` (Práctica 6)	Etapas, `junit`, `pollSCM` o webhook

```
<!-- pom.xml: el 0.80 es el mismo 80 % de la Seccion 7 del SQAP -->
<limit>
    <counter>LINE</counter>
    <value>COVEREDRATIO</value>
    <minimum>0.80</minimum>
</limit>
```

Evidencia 4: la tabla anterior completada para tu proyecto, con el fragmento de configuración de cada herramienta.

Paso 6. Estrategia de pruebas por nivel (Sección 7)

Para cada nivel de prueba define la técnica, la herramienta, el responsable y los criterios de entrada (cuándo se puede empezar) y de salida (cuándo se puede dar por terminado):

Nivel	Técnica y herramienta	Responsable	Criterio de entrada	Criterio de salida
Unitario	TDD, JUnit 5 + JaCoCo	Desarrolladores	Historia con criterios de aceptación	100 % de pruebas en verde; cobertura ≥ 80 % en módulos críticos
Integración	Pruebas de API con base de datos de prueba	Desarrolladores	Unitarias en verde	0 fallos; contratos de la API sin cambios no documentados
Sistema	E2E con Cypress; carga con JMeter	Tester	Build sano en staging	Escenarios críticos en verde; p95 dentro del objetivo
Aceptación	Escenarios del cliente, prueba de usabilidad	Líder + usuarios	Pruebas de sistema aprobadas	0 defectos S1 o S2 abiertos; firma del acta

Evidencia 5: la tabla de estrategia para tu proyecto, con la cobertura mínima global y por módulo crítico.

Paso 7. Gestión de defectos y SLA (Sección 8)

Define los estados de un defecto (nuevo, asignado, en progreso, resuelto, verificado, cerrado, reabierto), quién mueve cada transición y la diferencia entre **severidad** (cuánto daño causa, la decide QA) y **prioridad** (cuándo se corrige, la decide el líder). Luego fija el SLA:

Severidad	Definición	Respuesta	Resolución
Crítica (S1)	El sistema es inusable o hay pérdida de datos.	1 hora	8 horas

Severidad	Definición	Respuesta	Resolución
Alta (S2)	Funcionalidad principal afectada sin alternativa.	4 horas	24 horas
Media (S3)	Funcionalidad secundaria afectada o existe alternativa.	1 día hábil	1 semana
Baja (S4)	Defecto cosmético o mejora menor.	3 días hábiles	Siguiente sprint

> **Revisión rápida de un SLA:** leído de arriba hacia abajo, cada tiempo tiene que ser igual o mayor que el de la fila anterior. Si un defecto crítico tiene más plazo que uno alto, el SLA premia la gravedad. Y cada tiempo tiene que ser alcanzable por el equipo real: un SLA de 1 hora un domingo exige que alguien esté de guardia.

Evidencia 6: el diagrama de estados, la tabla de SLA y un defecto real o simulado de tu proyecto reportado con `plantilla-reporte-defecto.md`.

Paso 8. Riesgos de calidad (Sección 14)

En la hoja **Riesgos** califica la probabilidad y el impacto de 1 (bajo) a 3 (alto). La hoja calcula la exposición (probabilidad × impacto, de 1 a 9) y el nivel: 1–2 bajo, 3–4 medio, 6–9 alto. Cada riesgo de nivel alto necesita una mitigación **accionable**: una acción concreta, con responsable y momento, que reduzca la probabilidad o el impacto.

Riesgo	P	I	Exp.	Mitigación accionable	Resp.
Rotación de integrantes durante el semestre	3	3	9 Alto	Pair programming en módulos críticos; cada módulo documentado en el wiki antes de cerrar la historia.	Líder
Requerimientos ambiguos del cliente	3	2	6 Alto	Revisión del SRS con el cliente en la semana 1; prototipo validado antes de construir.	Arquitecto
Deuda técnica por presión de cronograma	2	3	6 Alto	Quality Gate bloqueante en el pipeline; 10 % de cada sprint para refactorizar.	SQA
Falla del servidor de integración continua	1	2	2 Bajo	Configuración de Jenkins en Git (`Jenkinsfile`, compose); procedimiento de reinstalación probado.	SQA
Librerías con vulnerabilidades críticas	2	3	6 Alto	Dependabot en cada PR; revisión mensual de dependencias.	Arquitecto

> **Mitigación accionable o no:** “tener cuidado”, “estar atentos” o “comunicarnos mejor” no son mitigaciones: no dicen qué hace quién ni cuándo. Prueba: si la mitigación no se puede poner como tarea en el tablero del equipo, no es accionable.

Evidencia 7: la hoja Riesgos con al menos cinco riesgos, todos los de nivel alto con mitigación accionable y responsable.

Paso 9. Revisar el SQAP mínimo contra la regla

Antes de terminar la clase, intercambien el SQAP mínimo con otro equipo. Cada equipo revisa el del otro contra los siete indicadores de la regla del plan verificable (Sección 3) y anota, por indicador, "cumple" o el texto exacto que lo incumple. Ese ejercicio es el entrenamiento para la Parte B: **revisar un plan es buscar las frases que no comprometen a nada**.

PARTE B — Reto guiado: el plan que no se puede verificar

Hasta aquí escribiste un plan. En esta parte recibes uno que escribió otro equipo: `SQAP-BIBLIOUSFX-v0.9.docx`, el plan de calidad del ejemplo conductor. El equipo anterior asegura que el plan "está completo y aprobado": tiene portada, índice, tablas, normas citadas y la firma del líder. **El documento tiene seis defectos sembrados.** Ninguno es de ortografía ni de formato: los seis hacen que alguna parte del plan no se pueda comprobar, y por lo tanto no proteja nada.

El plan trae como anexo el extracto de requerimientos no funcionales del SRS (RNF-01 a RNF-06). Úsalo: varios defectos solo se ven comparando lo que el plan promete con lo que el cliente pidió.

Paso 1. Leer el plan como auditor

Lee el plan completo una vez, sin anotar nada, para entender el proyecto. En la segunda lectura, recorre los siete indicadores de la regla del plan verificable **uno por uno**, de arriba hacia abajo, y para cada indicador recorre el plan entero. Es más lento que leer sección por sección, pero es lo que encuentra los defectos que viven entre dos secciones.

> **No corrijas lo que no está roto:** el plan tiene secciones correctas a propósito. Reportar como defecto algo que cumple la regla (por ejemplo, un SLA bien escalonado o un riesgo bien calculado) resta puntos igual que no encontrar un defecto real. Cada hallazgo debe citar el texto exacto y el indicador que incumple.

Paso 2. La tabla de auditoría

Antes de corregir nada, completa esta tabla en tu informe, una fila por defecto:

Columna	Qué escribir
Ubicación	Sección y tabla o párrafo del plan v0.9.
Evidencia textual	La frase o el valor exacto, entre comillas, tal como aparece.
Indicador violado	Cuál de las siete filas de la regla del plan verificable (Sección 3) incumple.
Consecuencia	Qué podría pasar en el proyecto real si el plan se aplica tal como está. Un ejemplo concreto.
Corrección	El texto antes y después. Si involucra una herramienta, la configuración que corresponde.
Entrada en el historial	La fila que agregas a la Sección 16 (versión, fecha, cambio, autor, revisor).

Paso 3. Método de trabajo: cuatro preguntas

Para cada compromiso del plan (un objetivo, una revisión, un umbral, un riesgo), responde en este orden y no saltes ninguna:

1. **¿Qué promete exactamente?** Cita la frase. No digas “la sección de métricas es floja”: di “la Sección 5 promete una interfaz intuitiva y amigable”.
2. **¿Cómo sabría si se cumplió?** Busca la métrica, la fórmula, el instrumento, las condiciones y el momento. Si falta alguno, ya tienes un hallazgo.
3. **¿Quién lo comprueba y dónde queda la evidencia?** Cruza con la Sección 3 (roles), la 6 (revisiones) y la 12 (registros). Si quien comprueba es quien produjo, o la evidencia no queda en ningún lado, ya tienes un hallazgo.
4. **¿Dicen lo mismo las demás secciones y las herramientas?** Busca el mismo número en todo el documento y en los fragmentos de configuración. Si aparece con dos valores, el plan se contradice: una de las dos versiones es la que va a ganar en la práctica, y casi siempre es la más permisiva.

> **Un defecto, una entrada en el historial:** igual que en la Práctica 6 cada defecto era un commit, aquí cada corrección es una fila en la Sección 16 del plan, con la versión 0.9.1, 0.9.2, etc. Si corriges seis cosas en una sola entrada “correcciones varias”, nadie puede saber después qué cambió ni por qué. La versión final corregida es la 1.0.

Paso 4. Un defecto resuelto — la métrica que no se puede medir

Este va resuelto como ejemplo del método. En la tabla de métricas de la Sección 5 del plan v0.9 aparece esta fila:

Característica	Métrica	Instrumento	Objetivo	RNF
Usabilidad	La interfaz debe ser intuitiva y amigable para los usuarios	Opinión del equipo	Alta	RNF-03

Pregunta 1 — ¿qué promete? Que la interfaz será “intuitiva y amigable”, con un nivel “alto”.

Pregunta 2 — ¿cómo sabría si se cumplió? No hay forma. “Intuitiva” no es una métrica: no tiene fórmula ni unidad, y “alta” no es un valor que se pueda comparar. El instrumento, “opinión del equipo”, lo mide quien construyó la interfaz, que es justamente quien menos dificultades va a encontrar en ella. Incumple **Medible** e **Instrumentado**.

Pregunta 3 — ¿quién y dónde? Nadie en particular, y no queda evidencia. Al final del proyecto el equipo dirá que la interfaz es intuitiva y el plan no le dará a nadie con qué discutirlo.

Pregunta 4 — ¿coincide con el resto? El anexo dice que RNF-03 pide que **un estudiante nuevo registre un préstamo sin ayuda**. Ese requerimiento ya trae la métrica escondida: la tasa de éxito de una tarea con usuarios nuevos. El plan la cambió por un adjetivo.

	Métrica	Instrumento y momento	Objetivo
Antes	La interfaz debe ser intuitiva y amigable para los usuarios	Opinión del equipo	Alta
Después	Tasa de éxito de la tarea “registrar un préstamo” sin ayuda; puntaje SUS	Prueba con 5 estudiantes que nunca usaron el sistema, al cierre del sprint 3; cuestionario SUS de 10 ítems	≥ 90 % de éxito; SUS promedio ≥ 68

Entrada en el historial: `0.9.1 | fecha | Sección 5: métrica de usabilidad medible y trazable a RNF-03 | autor | revisor`.

> **Lo que acaba de pasar:** este defecto no lo detecta ninguna herramienta de las Prácticas 1 a 6, porque ninguna herramienta mide adjetivos. Un pipeline puede estar en verde durante todo el semestre con una interfaz que ningún estudiante nuevo logra usar. Por eso la calidad se planifica antes: lo que no se escribe como número, no se verifica.

Paso 5. Los cinco defectos restantes

Cada uno incumple un indicador distinto de la regla. Estas pistas te dicen dónde mirar, no qué escribir.

Indicador	Pista
Asignado e independiente	Lee la tabla de roles de la Sección 3 junto con la tabla de revisiones de la Sección 6. ¿Quién escribe el código de los módulos críticos y quién aprueba sus pull requests?
Instrumentado	Hay una métrica con un número correcto y trazable, pero mira con qué, en qué condiciones y cuándo se mide. Compárala con lo que pide su RNF y con lo que aprendiste en la Práctica 5 sobre la carga.
Consistente	Busca el número de la cobertura mínima en todo el documento: en la Sección 5, en los criterios de salida de la Sección 7 y en los fragmentos de configuración de la Sección 9. ¿Cuál de los valores va a aplicar realmente el pipeline?
Completo	Cuenta las secciones y compáralas, una por una y en orden, con el Anexo A. Las secciones están numeradas sin saltos: no te fíes del número, fíjate en el título. Después mira cuántas dependencias de terceros declara la Sección 9.
Trazable	Recorre la matriz de riesgos multiplicando probabilidad por impacto. Una fila no da lo que dice. Y lee su mitigación con la prueba del tablero del Paso 8 de la Parte A.

Verificación de la Parte B: el plan corregido (versión 1.0) cumple los siete indicadores, y su Sección 16 tiene una entrada por cada uno de los seis defectos. Anota el dato importante: **no agregaste ninguna funcionalidad ni cambiaste el alcance del proyecto**. El plan se corrige haciendo verificables los compromisos que ya tenía, no prometiendo menos para que sea más fácil cumplir.

PARTE C — Ejercicios propuestos: el SQAP completo

Con el SQAP mínimo de la Parte A como base, el equipo completa las 16 secciones de IEEE 730-2002 en cuatro entregables parciales y un consolidado. Cada entregable agrupa secciones afines y recibe retroalimentación antes del siguiente: así se evita el patrón de "todo al final".

Entregable	Secciones IEEE 730-2002	Área de IEEE 730-2014	Plazo sugerido
Ejercicio 1 — Fundamentos	1, 2, 3, 15, 16	Implementación del proceso SQA	Semana 1
Ejercicio 2 — Aseguramiento del producto	4, 5, 7	Aseguramiento del producto	Semana 2

Entregable	Secciones IEEE 730-2002	Área de IEEE 730-2014	Plazo sugerido
Ejercicio 3 — Aseguramiento del proceso	6, 8, 9, 13	Aseguramiento del proceso	Semana 3
Ejercicio 4 — Gobernanza y riesgos	10, 11, 12, 14	Aseguramiento del proceso	Semana 3
Ejercicio 5 — Consolidado y defensa	Las 16	Las tres	Semana 4

Ejercicio 1 — Fundamentos del plan (Secciones 1, 2, 3, 15 y 16)

- Sección 1 de la Parte A revisada: proyecto, alcance delimitado, porción del ciclo de vida cubierta y nivel de integridad justificado.
- Documentos de referencia: SRS del proyecto, documento de arquitectura (SAD), normas adoptadas y acuerdos internos del equipo, con versión.
- Organigrama con roles explícitos y matriz RACI de las actividades de calidad. Justifica cómo se garantiza la independencia del Responsable SQA.
- Glosario con al menos 20 términos técnicos y del dominio del proyecto.
- Procedimiento de cambio del propio SQAP: quién propone, quién aprueba, cómo se versiona; tabla de historial con autor y revisor por sección.

Ejercicio 2 — Aseguramiento del producto (Secciones 4, 5 y 7)

- Documentos que el proyecto producirá (SRS, SAD, manual de usuario, manual técnico, informe de pruebas, actas de revisión): responsable, fecha prevista, criterio de adecuación y tipo de revisión de cada uno.
- Estándares de codificación adoptados y justificados para el stack del proyecto (Google Java Style Guide, PSR-12, PEP 8, Airbnb JavaScript Style Guide, Effective Dart) y convenciones de nombres para clases, variables, tablas, endpoints y pruebas.
- Hoja Métricas completa: al menos cinco características de ISO/IEC 25010 con dos métricas cada una, todas trazables a un RNF o a un riesgo, con instrumento, momento, objetivo y responsable.
- Estrategia de pruebas por nivel de la Parte A ampliada, con cobertura mínima global y por módulo crítico, y la tabla “el mismo número en dos lugares” con la configuración real de JaCoCo, el Quality Gate y JMeter.

Ejercicio 3 — Aseguramiento del proceso (Secciones 6, 8, 9 y 13)

- Calendario de revisiones en el ciclo de vida: revisión de requerimientos, de arquitectura, de diseño detallado, revisión de código en cada pull request y auditoría de cierre. Para cada una: quién, cuándo, qué artefacto, lista de verificación y criterio de aceptación.
- Inspección formal tipo Fagan (IEEE 1028) para al menos el SRS y el diseño de arquitectura: roles (moderador, autor, lector, inspectores, escriba), etapas y métricas de la inspección.

- Flujo completo de gestión de defectos: estados, transiciones permitidas y responsable de cada una; severidad × prioridad con SLA coherente; plantilla de reporte.
- Stack de herramientas: requerimientos, control de versiones, CI/CD, análisis estático, pruebas unitarias, E2E y de carga, gestión de defectos y comunicación, indicando qué práctica del curso cubre cada una.
- Plan de capacitación: competencias por rol, brechas detectadas en el equipo y sesiones planificadas.

Ejercicio 4 — Gobernanza y riesgos (Secciones 10, 11, 12 y 14)

- Control del repositorio: estrategia de ramas (GitHub Flow, Git Flow o trunk-based) justificada, protección de la rama principal, política de pull requests y de merge. Política de respaldos (qué, frecuencia, retención, responsable, prueba de restauración) y control de acceso.
- Control de proveedores: criterios de selección de dependencias (licencia, mantenimiento activo, comunidad), análisis de vulnerabilidades (Dependabot, OWASP Dependency-Check o Snyk) y revisión de licencias. Hoja Componentes de terceros con nombre, versión, licencia, justificación, responsable y fecha de la última revisión de seguridad.
- Catálogo de evidencias que se conservan: actas de revisión, informes de pruebas, registros del pipeline, informes de SonarQube y de JMeter; para cada una, ubicación, formato y período de retención.
- Hoja Riesgos con al menos 10 riesgos de calidad: probabilidad, impacto, exposición, mitigación accionable, responsable, disparador y plan de contingencia.

Ejercicio 5 — SQAP consolidado y defensa oral

Un solo documento en DOCX y PDF que integre los cuatro entregables ya corregidos según la retroalimentación, con portada, índice, numeración consistente, lista de tablas y figuras, y anexos. Acompañado de:

- El repositorio del proyecto piloto con el SQAP versionado en `docs/`, el libro `matrices-sqap.xlsx` actualizado y la plantilla de reporte de defectos.
- Una defensa oral de 20 minutos en la que el equipo justifica sus decisiones clave: por qué esas características de calidad, por qué esos umbrales, cómo se garantiza la independencia y qué riesgo les preocupa más.
- Una autoevaluación del plan contra los siete indicadores de la regla del plan verificable, hecha por un integrante que no redactó la sección evaluada.

Ejercicio 6 — Análisis escrito

Media página por pregunta, con ejemplos concretos de tu propio trabajo en esta práctica:

1. En el reto, el plan v0.9 tenía portada, índice, normas citadas y firma. ¿Por qué un documento así puede pasar una revisión superficial? ¿Qué indicador de la regla del plan verificable habría encontrado primero cada defecto?
2. De los seis defectos del reto, ¿cuáles habría detectado alguna herramienta de las Prácticas 1 a 6 durante el proyecto, y cuáles solo se ven leyendo el plan? ¿Por qué?
3. Elige un umbral de tu SQAP (cobertura, p95, deuda técnica). ¿Cómo justificas ese número y no otro? ¿Qué pasaría si el equipo lo baja a mitad del proyecto para que el pipeline vuelva a verde? Relaciónalo con lo discutido en las Prácticas 3, 5 y 6 sobre apagar reglas y subir umbrales.

4. IEEE 730 pide independencia del rol SQA. En un equipo de tres personas, ¿cómo la garantizaste? ¿Qué se pierde y qué se gana con una independencia solo parcial?

4. Entregable

Sube a Classroom, dentro de la carpeta `practica7` del repositorio del proyecto piloto:

- **Partes A y B (al final del segundo periodo de laboratorio):** PDF con las Evidencias 1 a 7 (alcance y nivel de integridad, RACI, métricas, el mismo número en dos lugares, estrategia de pruebas, SLA y reporte de defecto, riesgos) y la tabla de auditoría de seis filas del reto, con el plan BIBLIOUSFX corregido a la versión 1.0.
- **Parte C, Ejercicios 1 a 4:** cada entregable parcial en su plazo, como PDF y como commit en `docs/`.
- **Parte C, Ejercicio 5:** el SQAP consolidado (DOCX y PDF), `matrices-sqap.xlsx`, la plantilla de reporte de defectos, la autoevaluación y la presentación de la defensa oral.
- **Ejercicio 6:** el análisis escrito, en el mismo PDF del consolidado como anexo.

> **Verificación final:** antes de entregar, toma tres números de tu SQAP (por ejemplo, la cobertura mínima, el p95 y la deuda técnica) y búscalos en todo el documento y en la configuración de tus herramientas. Si alguno aparece con dos valores distintos, el plan todavía se contradice. Y lee en voz alta cada objetivo de la Sección 5: si alguno se puede cumplir “opinando”, todavía no es una métrica.

5. Rúbrica de evaluación

Criterio	Puntos
Parte A: SQAP mínimo con las Evidencias 1 a 7, revisado por otro equipo	15
Parte B: los 6 defectos del reto detectados y corregidos, sin reducir el alcance, con una entrada por defecto en el historial	15
Parte B: tabla de auditoría completa, con evidencia textual, indicador y consecuencia	5
Completitud estructural: las 16 secciones de IEEE 730-2002 con el contenido mínimo exigido	15
Atributos y métricas de ISO/IEC 25010: medibles, instrumentadas, trazables a requerimientos o riesgos	15
Plan de revisiones y estrategia de pruebas con listas de verificación y criterios de entrada y salida	10
Gestión de defectos con estados, severidad, prioridad y SLA coherente	5
Matriz de riesgos con al menos 10 riesgos y mitigaciones accionables	5
Integración con las Prácticas 1 a 6: el mismo umbral en el plan y en la configuración de las herramientas	5
Defensa oral	5
Ejercicio 6 y presentación profesional: redacción, numeración, normas citadas correctamente	5
TOTAL	**100**

Cada criterio se califica según cuatro niveles de logro:

Nivel	% del puntaje	Descripción
Excelente	90-100 %	Supera lo esperado, con rigor y decisiones justificadas.
Satisfactorio	70-89 %	Cumple lo esperado, con detalles menores corregibles.
En desarrollo	50-69 %	Contenido presente pero incompleto o parcialmente inadecuado.
Insuficiente	0-49 %	Contenido ausente, inadecuado o copiado sin adaptación al proyecto.

> **Cómo se pierden puntos rápido:** métricas sin número (“alta”, “rápido”, “intuitiva”), el mismo umbral con dos valores en el documento, el Responsable SQA aprobando su propio código, secciones omitidas en lugar de marcadas “no aplica” con justificación, riesgos con mitigaciones del tipo “estar atentos”, copiar el ejemplo BIBLIOUSFX cambiando solo el nombre, o corregir el reto bajando las exigencias del plan.

6. Errores frecuentes al redactar un SQAP

Síntoma	Causa probable y solución
El plan copia la norma: secciones llenas de definiciones de IEEE 730.	La norma dice qué preguntas responder, no las respuestas. Cada sección debe hablar de tu proyecto: nombres, módulos, números, fechas.
Una sección dice solo “No aplica”.	Falta la justificación. Escribe por qué no aplica a este proyecto. Si no puedes justificarlo, probablemente sí aplica.
Objetivos como “alta calidad”, “buen rendimiento”, “seguro”.	No son medibles. Recorre la cadena de la Sección 3 hasta llegar a un número con instrumento y momento.
Las métricas copian los valores de otro proyecto.	Cada umbral se justifica por el requerimiento o el riesgo de tu proyecto. Si no puedes decir por qué 80 % y no 60 %, todavía no lo decidiste.
La cobertura dice 80 % en una sección y 70 % en otra.	Define cada umbral en un solo lugar (la tabla de métricas) y en las demás secciones refiérete a esa fila.
El SLA no se puede cumplir con el equipo real.	Un SLA de 1 hora exige alguien disponible. Ajusta los tiempos a la disponibilidad real del equipo y escríbela.
Todos los riesgos son técnicos.	Incluye riesgos de personas (rotación, disponibilidad), de requerimientos, de proveedores y de herramientas.
El rol SQA lo tiene “todo el equipo”.	Si todos son responsables, nadie lo es. Un nombre, con independencia de los módulos críticos.
El plan se escribió al final, describiendo lo que ya se hizo.	Un SQAP es un compromiso previo. Si el proyecto ya existe, escribe el plan para la próxima versión y registra en el historial desde cuándo rige.

Síntoma	Causa probable y solución
No hay historial o dice “versión final”.	El SQAP cambia con el proyecto. Cada cambio es una fila en la Sección 16, con autor y revisor.

Anexo A — Las 16 secciones del SQAP según IEEE Std 730-2002

N.º	Sección	Qué debe responder
1	Propósito	Qué software cubre el plan, alcance, porción del ciclo de vida, nivel de integridad.
2	Documentos de referencia	Normas aplicables, SRS, documento de arquitectura, contratos, regulaciones.
3	Gestión	Organización, roles SQA, responsabilidades, independencia del rol de calidad.
4	Documentación	Documentos a producir, calendario de revisión, criterios de adecuación.
5	Estándares, prácticas, convenciones y métricas	Estándares de código, convenciones, métricas con valores objetivo y forma de monitoreo.
6	Revisiones de software	Revisiones técnicas, inspecciones, walkthroughs y auditorías.
7	Pruebas	Estrategia y niveles de prueba, criterios de entrada y salida, cobertura.
8	Reporte de problemas y acción correctiva	Flujo de defectos, clasificación, SLA, herramienta de gestión.
9	Herramientas, técnicas y metodologías	Herramientas de desarrollo, QA y CI/CD; técnicas como TDD, BDD o pair programming.
10	Control de medios	Repositorios, respaldos, acceso físico y lógico.
11	Control de proveedores	Calidad de librerías y componentes de terceros, seguridad de la cadena de suministro.
12	Registros: recolección, mantenimiento y retención	Evidencias a conservar, período de retención, ubicación.
13	Capacitación	Formación necesaria del equipo en herramientas, estándares y dominio.
14	Gestión de riesgos	Identificación, análisis, mitigación y monitoreo de riesgos de calidad.
15	Glosario	Términos del proyecto y del dominio.
16	Procedimiento de cambio e historia del SQAP	Control de versiones del propio plan.

Anexo B — Características de ISO/IEC 25010 y métricas de ejemplo

Característica (2011)	Subcaracterísticas	Métrica de ejemplo	Instrumento del curso
Adecuación funcional	Completitud, corrección, pertinencia	% de casos de aceptación que pasan; defectos funcionales por historia	JUnit, Cypress
Eficiencia de desempeño	Comportamiento temporal, uso de recursos, capacidad	Percentil 95 del tiempo de respuesta; throughput; uso de CPU bajo carga	JMeter
Compatibilidad	Coexistencia, interoperabilidad	% de contratos de API que validan contra OpenAPI; navegadores soportados	Pruebas de API, Cypress
Usabilidad	Aprendizaje, operabilidad, protección contra errores, accesibilidad	Tasa de éxito de tareas; SUS; errores de accesibilidad	Prueba con usuarios, Lighthouse
Fiabilidad	Madurez, disponibilidad, tolerancia a fallos, recuperabilidad	Densidad de defectos; disponibilidad; RTO	Gestor de defectos, monitor externo
Seguridad	Confidencialidad, integridad, no repudio, autenticidad, responsabilidad	Vulnerabilidades altas o críticas; Security Rating	SonarQube, Dependabot, OWASP ZAP
Mantenibilidad	Modularidad, reusabilidad, analizabilidad, modificabilidad, capacidad de prueba	Cobertura; deuda técnica; duplicación; complejidad ciclomática	JaCoCo, SonarQube
Portabilidad	Adaptabilidad, instalabilidad, reemplazabilidad	Tiempo de instalación en un ambiente limpio; build reproducible	Docker, Jenkins

La edición 2023 de ISO/IEC 25010 agrega la característica **protección** (safety) y renombra usabilidad como **capacidad de interacción** y portabilidad como **flexibilidad**. Cualquiera de las dos ediciones es aceptable si se indica cuál se usa en la Sección 2.

Anexo C — Normas y bibliografía

Normas

- IEEE Std 730-2014 — Software Quality Assurance Processes. IEEE Xplore.
- IEEE Std 730-2002 — Software Quality Assurance Plans.
- ISO/IEC 25010:2011 y 2023 — Systems and software Quality Requirements and Evaluation (SQuaRE) — Product quality model.
- ISO/IEC/IEEE 12207:2017 — Software life cycle processes.

- ISO/IEC 29110 — Lifecycle profiles for Very Small Entities (VSEs).
- IEEE Std 1012 — System, Software, and Hardware Verification and Validation (niveles de integridad).
- IEEE Std 1028 — Software Reviews and Audits.
- ISO/IEC/IEEE 29119 — Software testing.

Plantillas y bibliografía

- Plantilla SQAP de la Universidad de Kansas (H. Saiedian), basada en IEEE 730: people.eecs.ku.edu/~saiedian/Teaching/Stds/0730.pdf
- Pressman, R. S. y Maxim, B. R. (2020). Software Engineering: A Practitioner's Approach (9.ª ed.). McGraw-Hill. Parte sobre calidad y SQA.
- Galin, D. (2018). Software Quality: Concepts and Practice. Wiley-IEEE Press.
- Sommerville, I. (2015). Software Engineering (10.ª ed.). Pearson. Capítulo 24, Quality management.
- IEEE Computer Society (2024). SWEBOK Guide V4.0. Capítulo 12, Software Quality.
- ISTQB (2023). Certified Tester Foundation Level Syllabus v4.0.

El SQAP que escribiste hoy es el documento rector del resto del curso. En la Práctica 8 el pipeline de integración y despliegue continuo hará cumplir, en cada cambio, los umbrales que fijaste aquí: cada número de tu plan tiene que poder convertirse en una etapa que se pone roja cuando no se cumple.